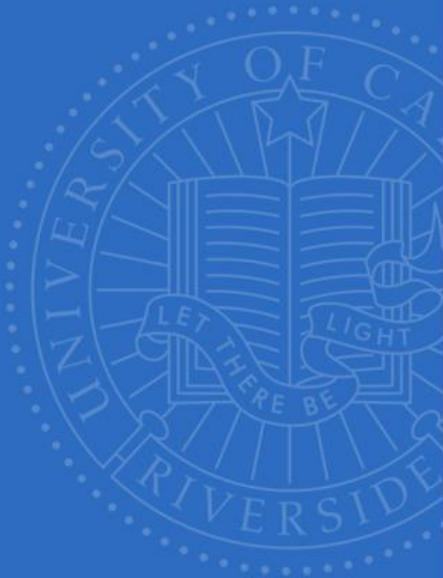


UCR



Project Ideas

UNIVERSITY OF CALIFORNIA, RIVERSIDE

**ANY APPLICATION YOU CAN THINK OF
THAT CAN BE MADE FASTER ON GPUS**

DATABASE/ANALYTICS

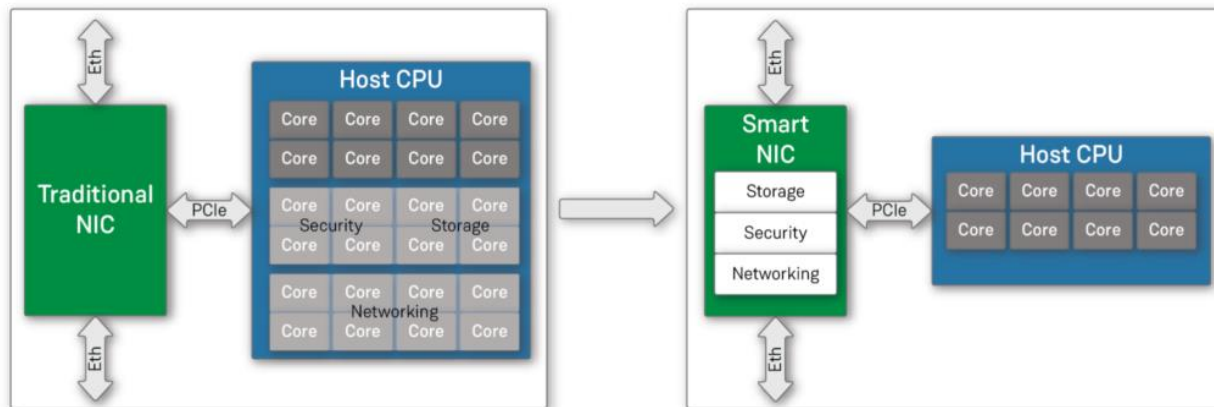
GPU-Accelerated Database Operators



- › Implement GPU versions of:
 - › Hash join
 - › Group-by aggregation
 - › Sort (radix sort)
- › Systems like Snowflake, Databricks, and NVIDIA RAPIDS rely on this
- › **Concepts:**
 - › Memory coalescing vs random access
 - › Partitioning strategies
 - › Load balancing across threads
- › **Stretch goal:**
 - › Compare CPU vs GPU performance on large datasets

GPU Network Packet Processing (Mini DPDK)

- Simulate or implement packet processing on GPU:
 - Packet parsing
 - Filtering (firewall rules)
 - Flow aggregation
- GPUs being explored for network offload (SmartNIC)
- **Concepts:**
 - Streaming data processing
 - Batching vs latency tradeoffs



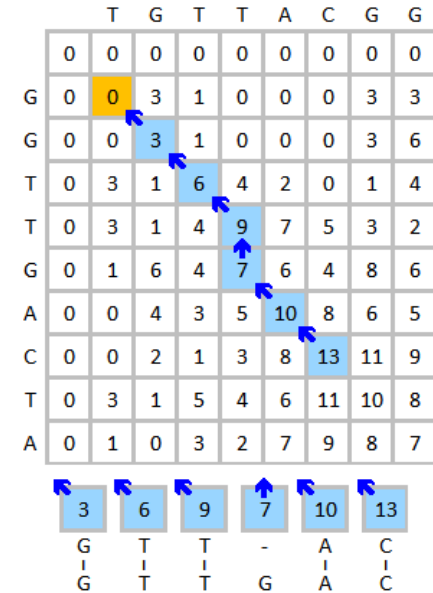
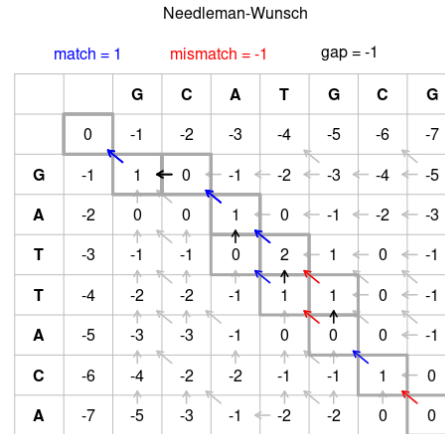
GPU-Based Log Processing / Search Engine



- › Implement parallel search over large logs
- › Keyword matching / regex-like filtering
- › Ranking results
- › **Stretch goal:**
 - › Add TF-IDF or embedding similarity search

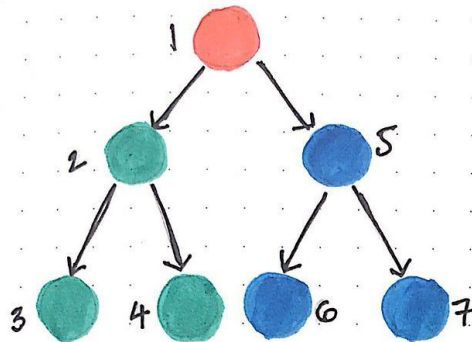
Bioinformatics: Sequence Alignment (GPU BLAST-lite)

- Needleman–Wunsch (global alignment)
- Smith–Waterman (local alignment)
- **Concepts:**
 - Dynamic programming on GPUs
 - Wavefront parallelism



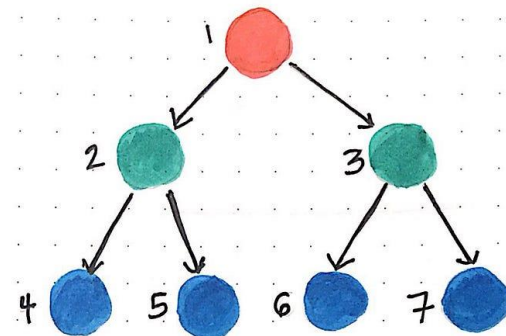
GPU Graph Processing Engine

- › Implement:
 - › BFS / DFS
 - › PageRank
 - › Connected components
- › **Concepts:**
 - › Frontier-based execution
 - › Warp divergence
- › **Stretch:**
 - › Compare push vs pull traversal



Depth-first search

- Traverse through left subtree(s) first, then traverse through the right subtree(s).

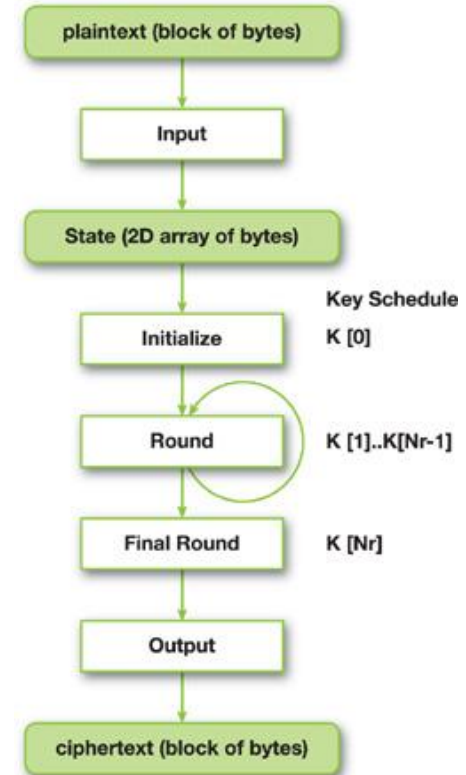


Breadth-first search

- Traverse through one level of children nodes, then traverse through the level of grandchildren nodes (and so on...).

GPU Cryptography (Hashing / Encryption)

- > Implement:
 - > SHA-256
 - > AES encryption
 - > Password brute-force simulator
- > **Concepts:**
 - > Bitwise operations
 - > Throughput optimization

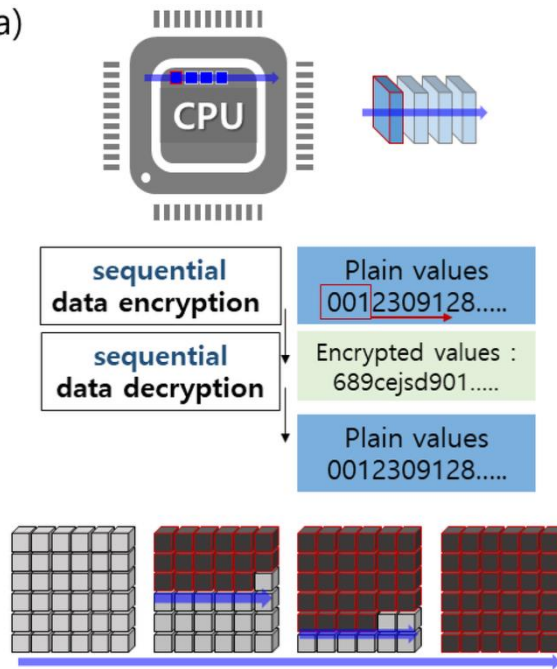


GPU Compression / Decompression Engine

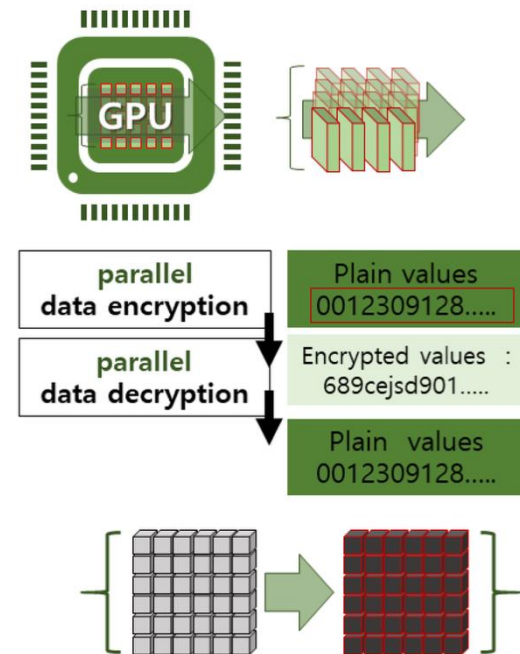
- Implement:
 - Run-length encoding (RLE)
 - LZ-style compression
 - Parallel decompression

- **Concepts:**
 - Parallel prefix sum
 - Data dependencies

(a)

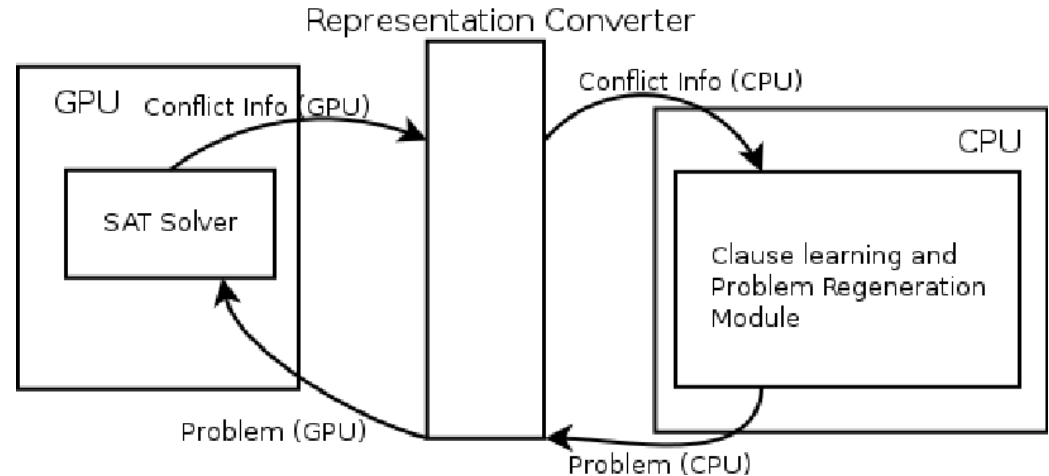


(b)



Parallel SAT Solver / Constraint Solver

- Implement:
 - SAT solving (DPLL/CDCL) simplified
 - Parallel exploration of search space



GRAPHICS

GPU Ray Tracer (Progressive Path Tracing)

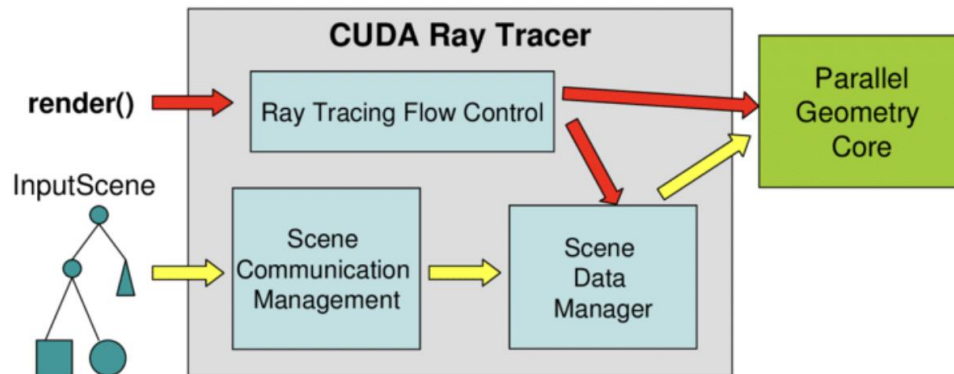
- › Implement:
 - › Start: spheres + basic lighting
 - › Add: reflections, shadows
 - › Extend: Monte Carlo path tracing (global illumination)

› Concepts:

- › Thread-per-pixel model
- › Recursion > iterative reformulation
- › Random sampling on GPU

› Stretch:

- › BVH (bounding volume hierarchy)
- › Denoising



Real-Time Volumetric Rendering (Fog/Smoke)

- › Implement **ray marching** through a 3D volume:

- › Render fog, smoke, or clouds
- › Simulate light scattering

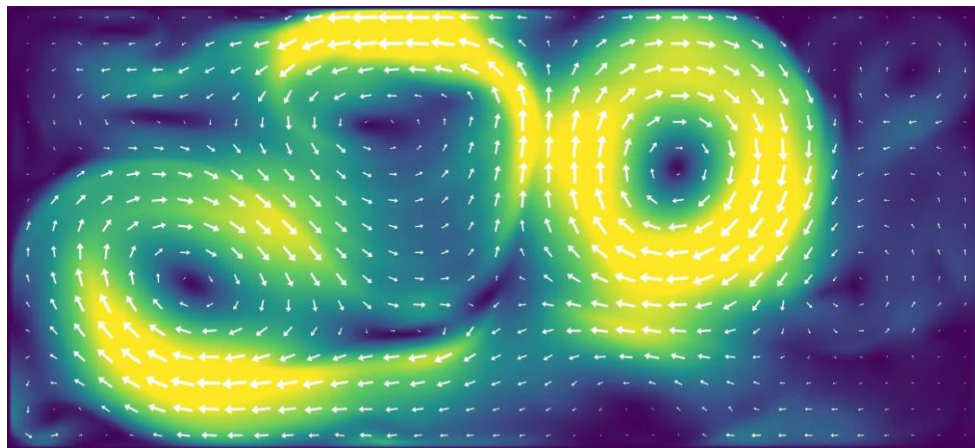
- › **Concepts:**

- › Sampling along rays
- › Tradeoff: quality vs performance
- › Early ray termination



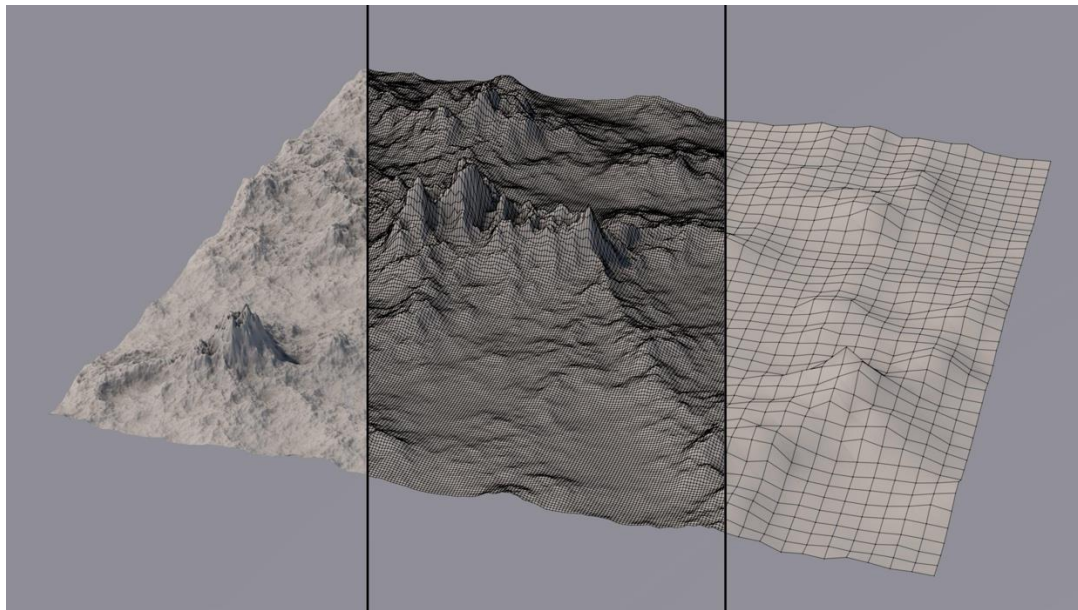
Real-Time Fluid Simulation (Stable Fluids)

- › Implement a 2D fluid solver:
 - › Velocity + pressure grids
 - › Advection, diffusion, projection
- › **Concepts:**
 - › Stencil computations
 - › Iterative solvers (Jacobi / Gauss-Seidel)
- › **Stretch:**
 - › Add interaction (mouse forces)



GPU Terrain Rendering + LOD

- › Render large terrains:
 - › Use Perlin/Simplex noise
 - › Implement Level-of-Detail (LOD)
- › **Stretch:**
 - › Clipmaps or chunked LOD
 - › Add erosion simulation



Neural Rendering (Tiny NeRF / Instant-NGP Lite)

- › Implement a simplified **NeRF**:
 - › Rays $\rightarrow$ sample points $\rightarrow$ MLP $\rightarrow$ color
 - › GPU-accelerated inference
 - › Tiny NeRF (small dataset, small network)
- › **Stretch:**
 - › Hash grid encoding (Instant-NGP idea)

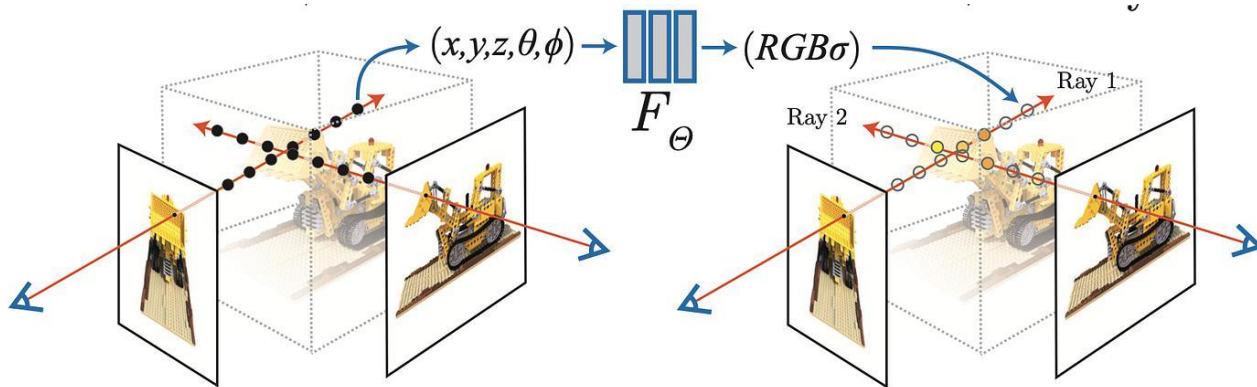
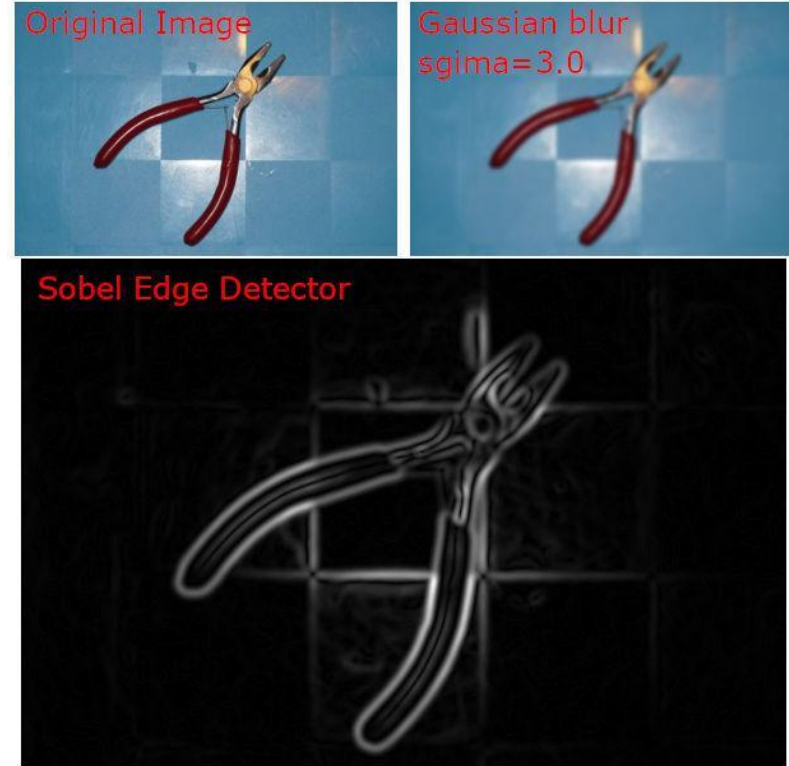


Image Processing Pipeline on GPU

- › Build a GPU image pipeline:
 - › Blur, sharpen, edge detect (Sobel)
 - › Color grading
 - › Chain filters
- › **Stretch:**
 - › Implement separable convolution optimization
 - › Real-time webcam filtering



MACHINE LEARNING / LLM

FlashAttention-style Kernel

› Implement a simplified version of **FlashAttention** from scratch:

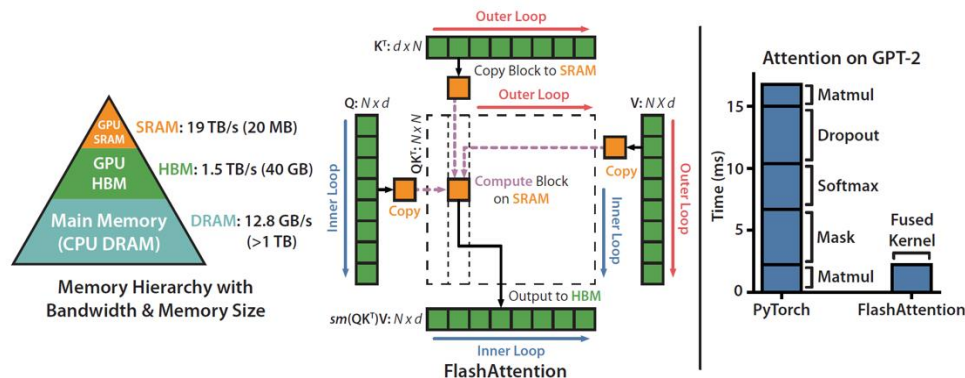
- › Baseline: naïve attention (PyTorch)
- › Optimized: tiled attention using shared memory (CUDA) or Triton
- › Optional: support causal masking

› **Concepts:**

- › Tiling, shared memory, coalescing
- › Warp-level reductions
- › Numerical stability (softmax)

› **Stretch:**

- › Compare with PyTorch 2.x fused kernels
- › Add BF16 / FP16 support



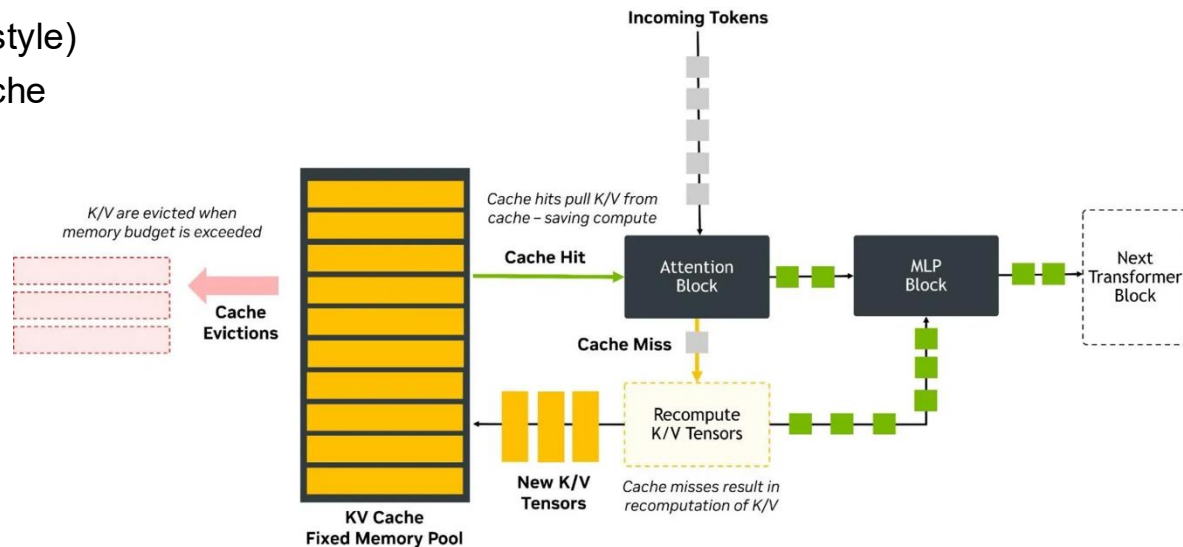
KV Cache Optimization for LLM Inference

- Implement and compare KV-cache strategies:

- Naïve append
- Paged KV cache (like vLLM-style)
- Chunked / sliding window cache

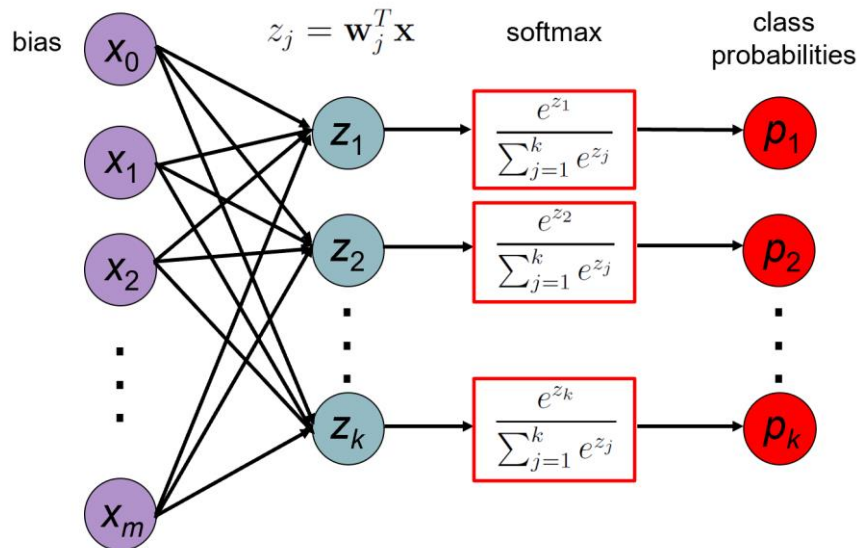
- Concepts:**

- Avoiding recomputation
- Optimized memory layout
- Paging and Batching



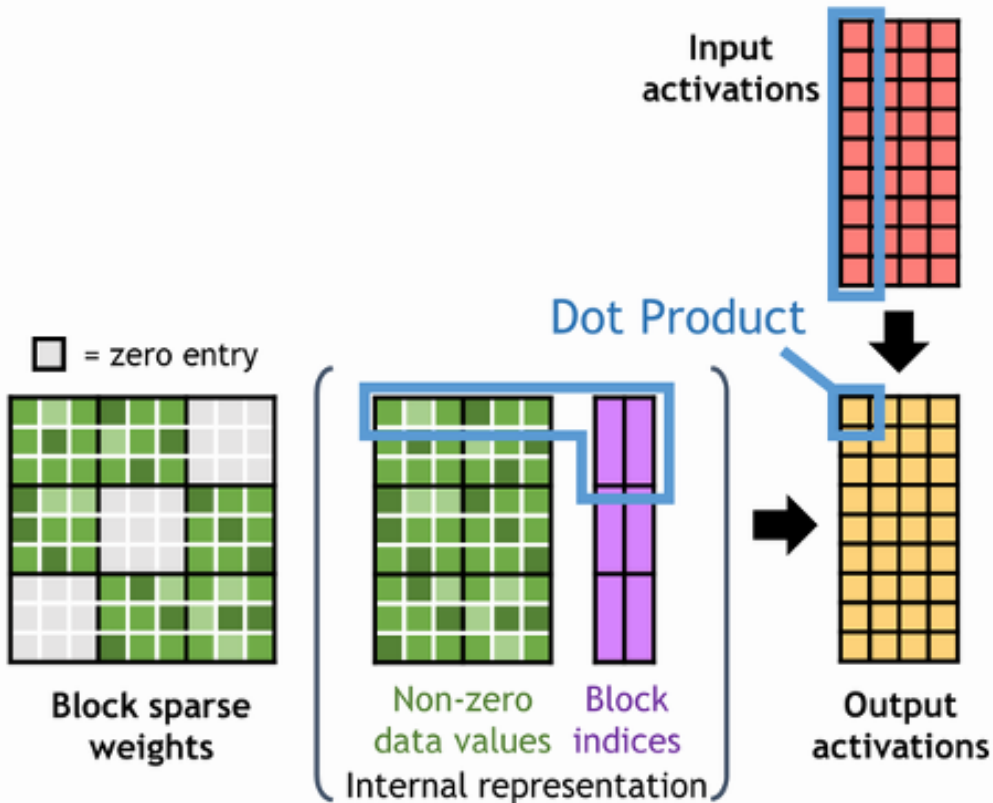
Custom GPU Kernel for Softmax / Reduction

- > Implement multiple softmax kernels:
 - > Naïve global memory version
 - > Shared memory reduction
 - > Warp-level reduction (shfl)
- > **Stretch:**
 - > Fuse softmax + scaling + masking



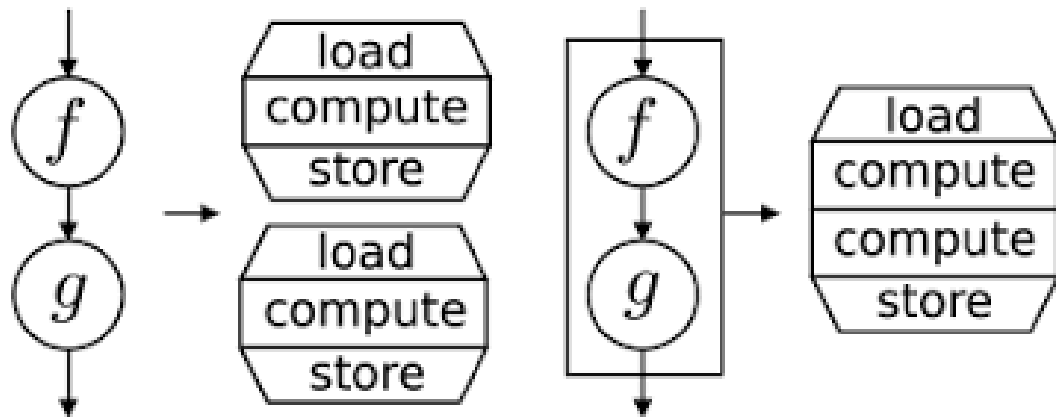
Sparse Matrix Multiply on GPU (SpMM)

- > Implement SpMM:
 - > CSR/COO formats
 - > Optimize memory access
 - > Compare with dense GEMM
- > **Concepts:**
 - > Load Imbalance
 - > Warp divergence
 - > Memory irregularity



Operator Fusion Engine (Mini Compiler)

- > Build a simple graph optimizer:
 - > Input: PyTorch model graph
 - > Fuse:
 - > matmul + bias + activation
 - > softmax + scaling
- > Generate:
 - > Triton kernels OR
 - > TVM schedule



Mixed Precision + Quantization Kernel Study

- Implement kernels for:
 - FP32 vs FP16 vs BF16
 - INT8 quantized matmul
- Measure:
 - Accuracy loss
 - Speedup
 - Memory savings

